IT342 Phase 2 – Mobile Development

Project: Knock Knock

Student: Niña Nicole S. Villadarez

Course: IT342

---

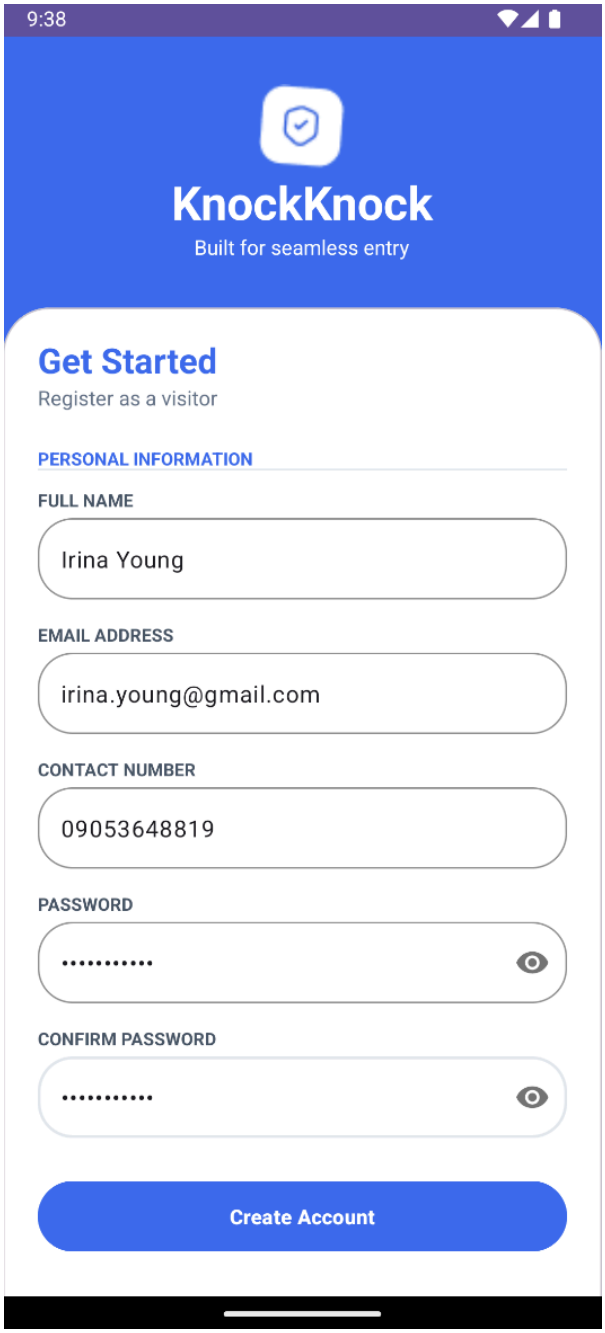
1. GitHub Repository Link
Repository Link: https://github.com/theninanicole/IT342_KnockKnock_G4_Villadarez

2. Final Commit for Phase 1
Final commit message: IT342 Phase 2 – Mobile Development Completed

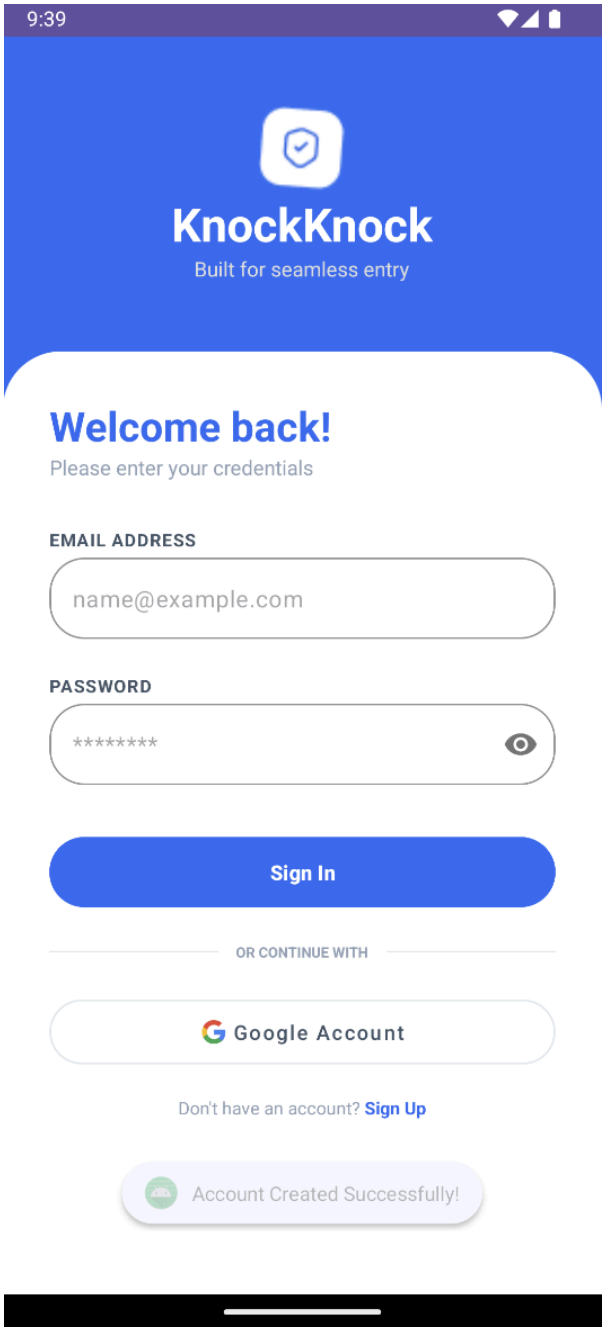
Commit Link / Hash:
https://github.com/theninanicole/IT342_KnockKnock_G4_Villadarez/commit/4c3e9ce3e49e58966beae39cc39a4042bda3ed10

3. Screenshots of Implementation

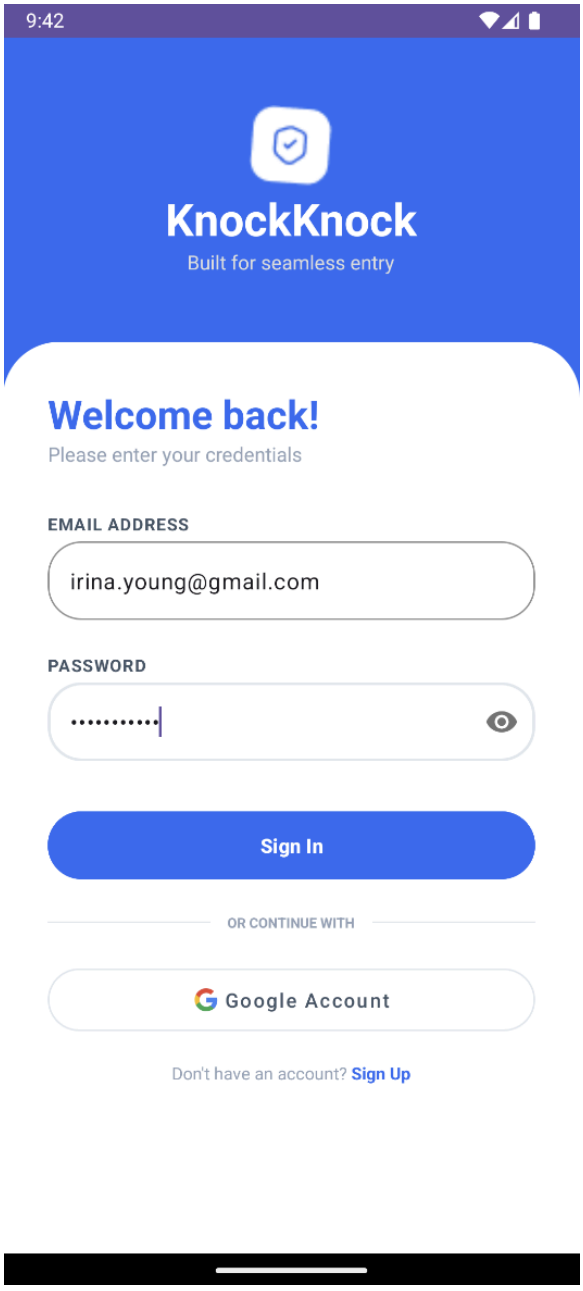
Registration Screen



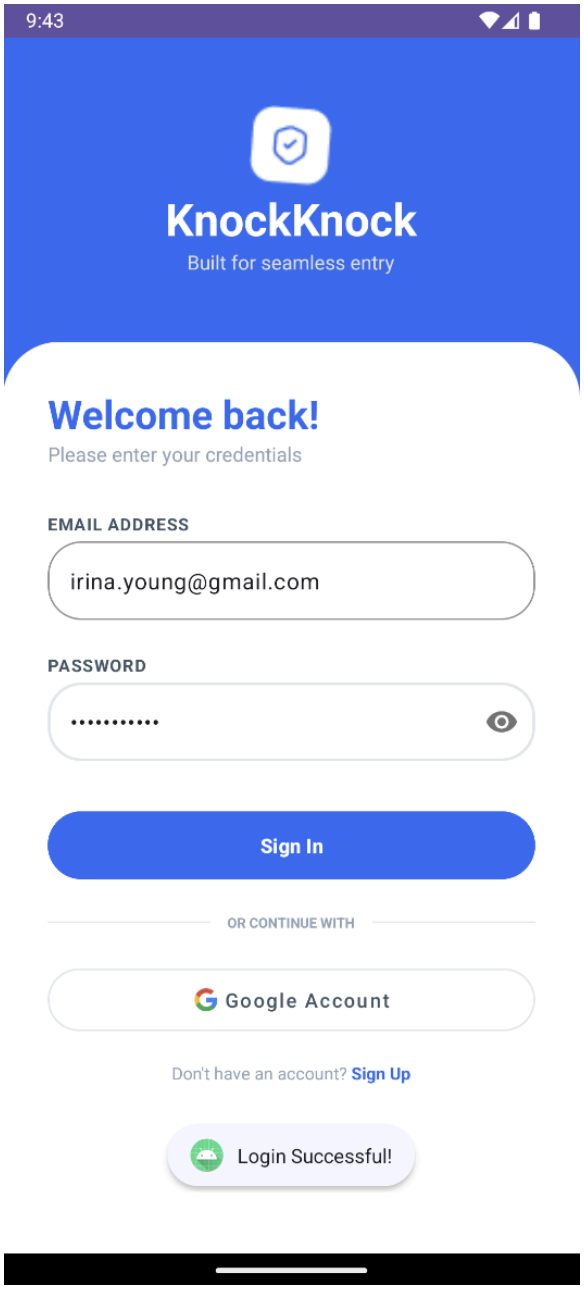
Successful Registration



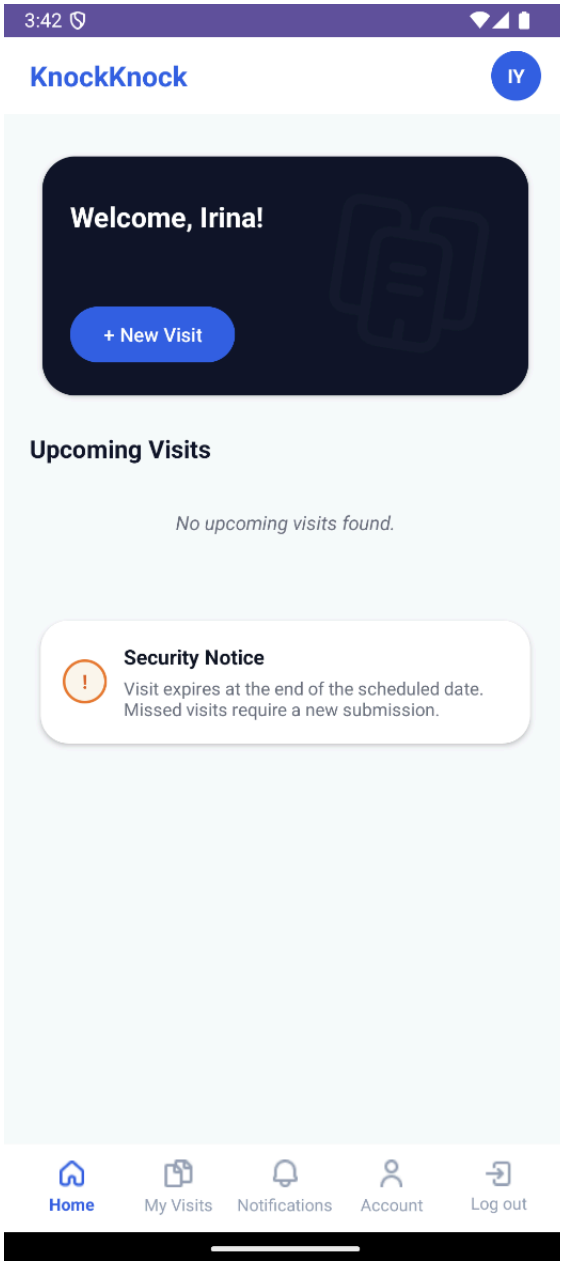
Login Screen



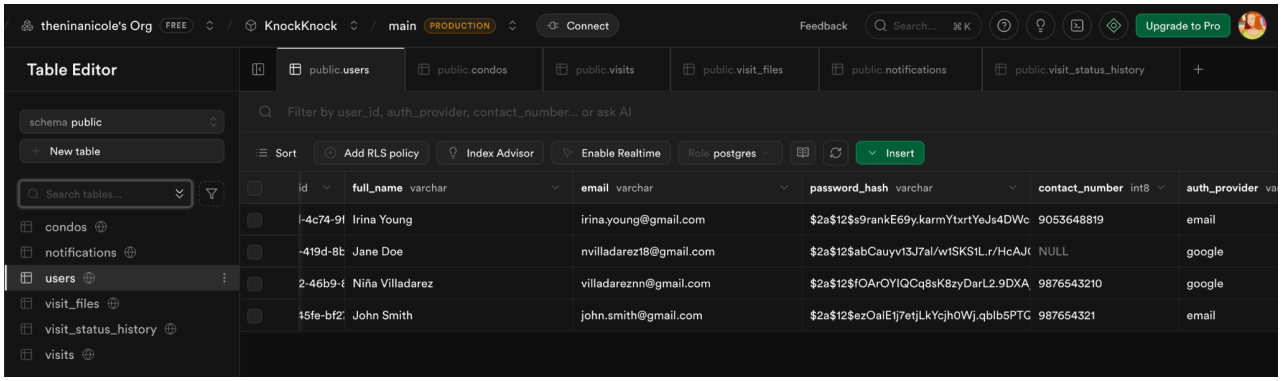
Successful Login



After Login Screen



Database Record



4. Implementation Summary

I. How Registration Works

- The Android app provides a registration screen where users can create either a visitor account or a condominium admin account.
- The form collects the following core fields: full name, email address, contact number, password, and confirm password. For condominium admins, it also asks for condominium name and condominium address.
- On submit, the app performs basic client-side checks (required fields, valid-looking email, matching password and confirm password). If any check fails, the user is prompted to correct the input before the request is sent.
- When the form is valid, the app sends the data to the backend via Retrofit:
    - Visitor: `POST /api/auth/register/visitor`
    - Condominium Admin: `POST /api/auth/register/condo-admin`
- The backend handles full validation (including duplicate-email checks and secure password hashing) and returns an AuthResponse containing the created user and a JWT token if registration succeeds, or an error message if it fails. The app displays the toast message and then redirects the user to login.

II. How Login Works

- The login screen accepts an email address and password from the user.
- When the user taps the “Sign In” button, the app creates a LoginRequest and calls `POST /api/auth/login` via Retrofit.
- The backend verifies the credentials by:
    - Looking up the user by email.
    - Comparing the provided password with the securely stored hashed password.
- If the credentials are correct, the backend responds with an AuthResponse containing:
    - The authenticated user (id, fullName, email, role, and optional condo info).
    - A signed JWT token.
- The app saves the JWT in SessionManager (a SharedPreferences helper), which persists the token on the device.

- After login, the app reads the user's role and routes them accordingly. For visitors, it opens VisitorDashboardActivity, which is the main home screen for visitor accounts.
- On subsequent launches, the app checks if a token exists:
    - If present, it can call `GET /api/auth/me` to verify the session and skip the login screen.
    - If missing or invalid, the user is redirected back to login.
- Logging out clears the stored token and navigates the user back to the login screen.

III. API Endpoints

The mobile app uses Retrofit with a common base URL (http://10.0.2.2:8080/api/) to communicate with the Spring Boot backend. All requests are made asynchronously with Kotlin coroutines.

Authentication APIs:

`POST /api/auth/register/visitor` – Register new visitor accounts from the mobile app.

`POST /api/auth/register/condo-admin` – Register new condominium admin accounts with condominium details.

`POST /api/auth/login` – Authenticate users by email and password and return a JWT token.

`GET /api/auth/me` – Fetch the currently authenticated user using Authorization: Bearer <token>; used by the visitor dashboard to load the user profile, generate initials for the avatar, and confirm the VISITOR role.